

OPTIMISE@Wits

Annual Unconstrained Optimisation Challenge

School of Computer Science and Applied Mathematics, University of the Witwatersrand

The challenge

Design one reliable numerical solver for a collection of hidden, smooth, unconstrained optimisation problems. Your solver must use only the information provided through the official problem interface and must work without knowing the formula, minimiser, conditioning, or geometry of each hidden objective.

1 Purpose and learning outcomes

The tournament is a capstone activity for APPM2025A. It assesses whether a team can translate optimisation theory into a robust computational method. By the end, students should be able to:

- select and combine methods such as steepest descent, line search, Newton or quasi-Newton updates, random restarts, and direct search;
- diagnose poor scaling, narrow valleys, saddle behaviour, and non-convexity;
- manage a limited function-evaluation budget;
- justify algorithmic choices using evidence rather than trial-and-error alone.

2 Teams, eligibility, and timeline

Teams contain **three or four students**. Mixed-discipline teams are strongly encouraged. Every registered student must contribute to the design, testing, or analysis of the submitted solver.

Registration closes	Monday 31 August 2026, before class
Public leaderboard opens	Monday 31 August 2026, 08:00
Public leaderboard closes	Wednesday 30 September 2026, 23:59
Final evaluation opens	Thursday 1 October 2026, 08:00
Final submission deadline	Saturday 10 October 2026, 02:00

3 What each team must upload

Upload exactly one file: `solver.py`, the implementation of the required function `solve(problem)`, inside a `.zip` archive.

Do not upload notebooks, datasets, additional modules, executables, or external model files. The evaluator imports only `solver.py`. There is no report.

4 Required solver interface

```
def solve(problem):
    """Return one candidate minimiser for the supplied problem.

    Parameters
    -----
    problem : Problem
        Provides n, x0, budget, seed, value(x), and possibly gradient(x).

    Returns
    -----
    dict with keys:
        "x"      : one-dimensional NumPy array of length problem.n
        "status" : short string, for example "ok" or "budget_exhausted"
        "method" : short description of the method used
```

'''

Important

The evaluator recomputes the final objective value. A team gains no advantage by reporting its own objective value. The returned vector must be finite and have exactly the required dimension.

5 Competition rules

- The mathematical problems are unconstrained: each objective is defined on $\mathbb{R}^n$. A numerical safety limit rejects non-finite or astronomically large iterates.
- Only Python's standard library and NumPy may be imported. Calls to `scipy.optimize`, automatic differentiation libraries, internet services, external executables, or pretrained models are not permitted.
- The solver may use the supplied random seed for reproducibility.
- The evaluation budget is strict. Function and gradient calls are charged according to the API specification.
- File access, network access, process creation, and attempts to inspect hidden benchmark data are prohibited.
- Teams may submit several times during the public phase, subject to the announced submission limit. The final ranking uses unseen hidden instances.

6 Scoring and ranking

The tournament mark is **100% automated**: it is determined entirely by your solver's performance on the hidden benchmark suite. There is no report and no oral examination.

Each valid run receives a score based on objective reduction, evaluation efficiency, and robustness. The public leaderboard is informative only; it does not reveal the final hidden ranking.

The automated score rewards orders-of-magnitude reduction in the optimality gap and includes a small efficiency bonus. Invalid output or an exception scores zero on that instance. Tie-breaks are, in order: more high-accuracy solves, fewer evaluations, then earlier valid final submission.

7 Use of AI tools and academic integrity

AI tools may be used for brainstorming, explanations, or code review, but the submitted solver remains the responsibility of the team, and AI assistance must be declared on the submission form. Every member must understand the method well enough to explain it.

Copying another team's code, sharing a solver between teams, using information about the hidden instances, or attempting to inspect the evaluation engine constitutes academic misconduct. All submissions are screened automatically and compared against one another, and the code of leading teams is read by the course staff before marks are released.

Recommended strategy

Begin with a correct baseline, test it on every public instance, record failures, and improve one weakness at a time. A robust hybrid method usually outperforms a fragile method that is excellent on only one function family.

8 Recognition

Awards may include Overall Winner, Runner-up, Best Mathematical Insight, Most Robust Solver, and Best Cross-Disciplinary Team. The official course announcement will confirm prizes and the mark contribution.

9 Final submission checklist

Before submitting, every team should confirm that:

- the archive contains exactly one file, named `solver.py`, at its top level;

- `solver.py` imports only NumPy and standard-library modules;
- `solve(problem)` returns the required dictionary for every public instance;
- all random choices use `problem.seed`;
- the solver retains the best point found and handles budget exhaustion;
- all team members can explain the search direction, step-size rule, safeguards, and main failure mode.

What will not help

Hard-coding textbook minimisers, identifying a function by its first few values, using fixed coordinate assumptions, or tuning exclusively to the public suite is unlikely to generalise. Hidden instances use unseen shifts, rotations, dimensions, scales, and gradient availability.

What a strong submission looks like

A strong solver is valid on every instance, reduces the objective reliably, adapts when its first strategy stagnates, spends its budget deliberately, and is simple enough for the whole team to explain.